

lab9

YUE CHENGHAO

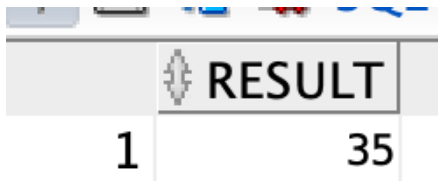
227154

Q1

- code

```
CREATE OR REPLACE FUNCTION ADD_TWO_VALUE (p_a NUMBER, p_b NUMBER)
RETURN NUMBER
IS
BEGIN
    RETURN p_a + p_b;
END;
/
SELECT ADD_TWO_VALUE(10, 25) AS result
FROM dual;
```

- output



	RESULT
1	35

Q2

- code

```
--Q2
CREATE OR REPLACE PROCEDURE raise_amount (
    p_emp_id    IN employees.employee_id%TYPE,
    p_raise_pct IN NUMBER
)
```

```

IS
BEGIN
    UPDATE employees
    SET salary = salary * (1 + p_raise_pct)
    WHERE employee_id = p_emp_id;

    COMMIT;
END;
/
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE employee_id = 143;

EXEC raise_amount(143, 0.10);

SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE employee_id = 143;

```

- output

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY
1	143	Randall	Matos	2600

Q3

- code

```

CREATE OR REPLACE PROCEDURE DisplayEmployeeInfo (EmpNum IN
employees.employee_id%TYPE)
IS
    v_name employees.first_name%TYPE;
    v_last employees.last_name%TYPE;
    v_job   employees.job_id%TYPE;
    v_sal   employees.salary%TYPE;
BEGIN
    SELECT first_name, last_name, job_id, salary
    INTO v_name, v_last, v_job, v_sal
    FROM employees
    WHERE employee_id = EmpNum;

```

```

DBMS_OUTPUT.PUT_LINE('Employee Name: ' || v_name || ' ' || v_last);
DBMS_OUTPUT.PUT_LINE('Employee position: ' || v_job);
DBMS_OUTPUT.PUT_LINE('Salary: ' || v_sal);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found for ID = ' || EmpNum);
END;
/
EXEC DisplayEmployeeInfo(143);

```

- output

Procedure DISPLAYEMPLOYEEINFO 已编译

PL/SQL 过程已成功完成。

```

Employee Name: Randall Matos
Employee position: ST_CLERK
Salary: 2860

```

Q4

- code

```

--Q4
CREATE OR REPLACE PROCEDURE InsertDepartment (
    p_dept_id    IN departments.department_id%TYPE,
    p_dept_name  IN departments.department_name%TYPE
)
IS
BEGIN
    INSERT INTO departments (department_id, department_name)
    VALUES (p_dept_id, p_dept_name);

    COMMIT;
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Department ID already exists: ' || p_dept_id);
END;

```

```

/
EXEC InsertDepartment(300, 'Computer Science');
SELECT department_id, department_name
FROM departments
WHERE department_id = 300;

```

- output

	DEPARTMEN...	DEPARTMENT_NAME
1	300	Computer Science

Q5

- code

```

--Q5
--create an audit form
CREATE TABLE AUDIT_LOG_SALARY (
    audit_id      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    employee_id   NUMBER,
    old_salary    NUMBER(8,2),
    new_salary    NUMBER(8,2),
    change_date   DATE,
    change_time   VARCHAR2(8)
);
--Rebuild trigger
CREATE OR REPLACE TRIGGER trg_audit_salary
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log_salary (employee_id, old_salary, new_salary,
    change_date, change_time)
    VALUES (
        :OLD.employee_id,
        :OLD.salary,
        :NEW.salary,
        TRUNC(SYSDATE),
        TO_CHAR(SYSTIMESTAMP, 'HH24:MI:SS')
    );

```

```

END;
/
--test
UPDATE employees
SET salary = salary + 500
WHERE employee_id = 143;

COMMIT;
--check result
SELECT employee_id, old_salary, new_salary, change_date, change_time
FROM audit_log_salary
WHERE employee_id = 143
ORDER BY audit_id DESC;

```

- output

	EMPLOYEE_ID	OLD_SALARY	NEW_SALARY	CHANGE_DATE	CHANGE_TIME
1	143	2860	3360	05/01/26	11:23:57

Q6

- code

```

--q6
CREATE OR REPLACE FUNCTION RETURN_TOTAL_SALARY (p_dept_id IN
employees.department_id%TYPE)
RETURN NUMBER
IS
    v_total NUMBER;
BEGIN
    SELECT NVL(SUM(salary), 0)
    INTO v_total
    FROM employees
    WHERE department_id = p_dept_id;

    RETURN v_total;
END;
/

```

```
SELECT RETURN_TOTAL_SALARY(50) AS total_salary_dept_50  
FROM dual;
```

- output

TOTAL_SALARY_DEPT_50	
1	18260